Taught compositions, untaught problems: what small models transfer when fine-tuned to write programs

Abstract

A small language model can be fine-tuned to translate word problems into executable programs, so that a runtime does the arithmetic. What such a model transfers to unseen problems determines what training data it needs. We report a fine-tuning programme of small open models (0.5B to 1.7B parameters) that write programs in a small dataflow language, together with evaluation-only experiments that test what its results mean. The fine-tuned models wrote valid programs almost always and solved problems whose solution program they had been trained on, but almost none from untrained problem types; recipe, adapter, data mixture, prompt, and retries did not change that. A curriculum of declared operation combinations gave complete transfer to unseen combinations, but rewording the same problems showed this transfer to depend on the training generator's phrasing: it broke trained combinations as well as held-out ones, held-out ones more, and least in the largest run. Book problems whose solution shape had been taught were solved once reworded in the teaching template, even by a run that solved none in their own wording. Untuned models writing plain JavaScript composed the elementary operations almost as well, and an untuned 1.7B model solved most problems of a type the fine-tuned models rarely solved. Two larger models prompted with the same language solved nearly every problem in every wording, locating these limits in the small models' capacity. For small program-writing models, fine-tuning buys reliable execution of taught shapes in taught phrasings, which suits bounded problem classes rather than broad coverage.

Keywords: small language models; program synthesis; compositional generalization; fine-tuning; synthetic data; curriculum design

1. Introduction

After three epochs of fine-tuning, a 0.5-billion-parameter code model wrote a syntactically valid program, with a valid dependency graph, for every one of 225 held-out problems. On validation problems whose solution program also appeared in training, those programs returned the right answer 99% of the time. On the held-out problems, drawn from problem types absent from training, they returned the right answer zero times. The model had learned the language completely and had learned no way to use it on something new.

This paper follows what happened next: a sequence of attempts to move that zero, what each attempt showed, and what finally changed the result and by how much. The setting is practical. Program-aided reasoning, in which a language model writes code and an interpreter executes it, moves arithmetic out of the model's job (Gao et al., 2023; Chen et al., 2023). Small models are attractive for this role because they are cheap to run and can be specialized. Fine-tuning on reasoning traces improves small models' task performance (Magister et al., 2023), and a small model's capacity can be concentrated on a target reasoning task at a cost to its general abilities (Fu et al., 2023). Anyone building such a system must decide what training data to generate: more instances of the same problems, more kinds of problems, different presentation, or a larger model. The right choice depends on what the model transfers from program-writing data.

Work on compositional generalization suggests the answer is structural. Lake and Baroni showed that sequence-to-sequence networks can fail to generalize systematically to new combinations of known

primitives (Lake & Baroni, 2018). Keysers and colleagues showed that such failures track how compounds, rather than atoms, differ between training and test (Keysers et al., 2020). Kim and Linzen found sequence-to-sequence models near-perfect in distribution and far weaker on novel combinations in a semantic-parsing benchmark (Kim & Linzen, 2020). Bogin and colleagues showed that test instances containing local structures unobserved in training are the hard ones (Bogin et al., 2022). In program synthesis specifically, Shi and colleagues found that neural synthesizers generalize poorly to compositions they were not trained on (Shi et al., 2022; Shi et al., 2024). Dziri and colleagues argued that transformers often reduce multi-step reasoning to matching linearized computation subgraphs seen in training (Dziri et al., 2023). These studies use purpose-built benchmarks or larger models. Our setting is a small model fine-tuned on synthetic programs for a practical task, and our question is operational: which changes to training moved transfer to unseen problem types, and which did not.

We make four contributions.

1. We document a coverage ladder, from trained programs through new programs of trained types to untrained types, and the failure mechanisms visible in the generated programs.
2. We report a sequence of interventions that left transfer to untrained types at or near zero, with what each one's own measurements showed.
3. We show what the one effective change, a declared inventory of operation combinations with whole combinations held out, did and did not achieve. Transfer within the inventory was complete in the training generator's wording and degraded when the same problems were reworded. Teaching missing solution shapes made them solvable in the teaching template but often not in the book's wording.
4. We compare the fine-tuned students with the untuned models they started from, prompted to answer directly, to write plain JavaScript, or to write the same language from examples. The comparison shows what fine-tuning adds and what it removes.

2. Setting

2.1 The target language

The models write programs in a small dataflow language developed for this project, SOP Lang. A program is a set of named computations called wires. Each wire is declared on a line `@name command` followed by a body, and a body can read another wire's value as `$name`. Those references define a dependency graph that the runtime executes in order. Two commands matter here: `literal`, whose body is a JSON value, and `jsEval`, whose body is JavaScript executed in a sandbox. A compiled solution records the quantities extracted from the problem in a `slots` literal and computes the answer from them:

```
@slots literal
{"packs": 3, "perPack": 4}

@answer jsEval
return $slots.packs * $slots.perPack;
```

The runtime returns the value of `answer`, here 12. The model reads the problem, extracts the values, and chooses and writes the computation; the runtime executes it exactly. Longer solutions add intermediate wires, for example a filtered list that a later wire totals.

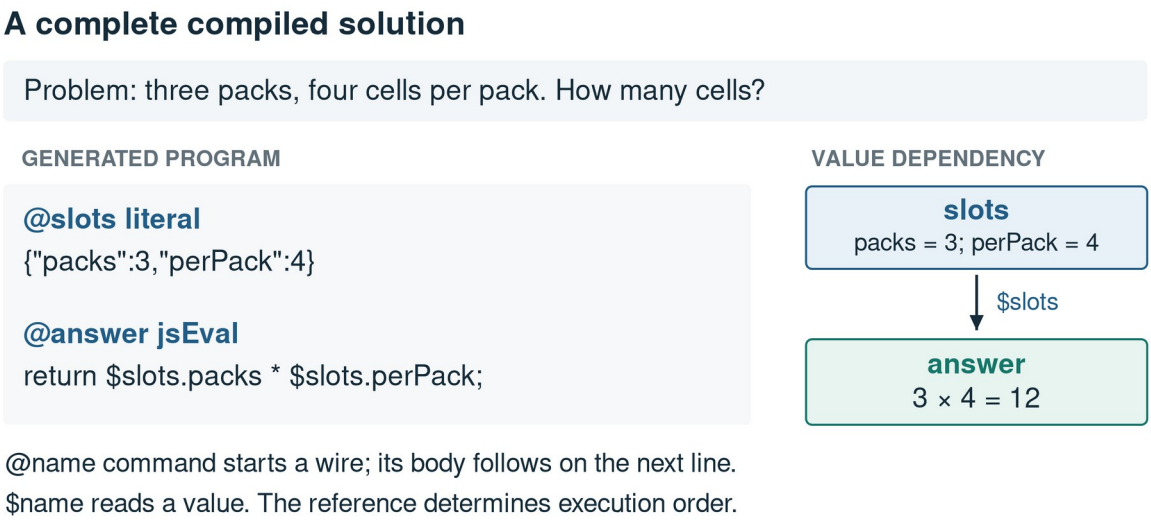


Fig. 1 Anatomy of a compiled solution. The model writes both wires; the runtime resolves the reference, executes the body, and returns 12. Constructed example

2.2 Problems, types, and program fingerprints

Training problems came from two sources. Seven books of reasoning exercises were transcribed into problem types. Each type generates statements from a printed template with a reference program and an independently computed answer. We call these book types. A procedural generator produced further types built from elementary operations on lists of numbers, such as filtering by a threshold, totalling, counting, taking the largest value, and adding a fixed amount. Every accepted example was executed, and its inputs were perturbed to confirm that the answer depended on them.

Each reference program carries a fingerprint: a hash of its source text with the problem's specific values removed. The text includes wire names, assertion messages, and the wording of the returned sentence. Two problems share a fingerprint when their reference programs are identical apart from values. A new fingerprint therefore marks a new program text; it may or may not be a new computation. We use fingerprints to separate three kinds of held-out problem: a trained program with new values, a new program within a trained type, and a problem from a type absent from training. The first dataset held 6,775 examples over 893 fingerprints, distributed very unevenly: one book contributed 579 fingerprints over 990 examples, another nine fingerprints with 100 examples each.

2.3 Models, training, and evaluation

The runs fine-tuned Qwen2.5-Coder-0.5B-Instruct (Qwen, n.d.-a), Qwen2.5-Coder-1.5B-Instruct (Qwen, n.d.-b), and Qwen3-1.7B (Qwen, n.d.-c); the last is a general-purpose model rather than a code model. Training was full fine-tuning with AdamW at learning rate 10^{-4}, effective batch size 32, sequence length 4,096, and a fixed seed. Early runs trained for three epochs and later runs for two. Checkpoints were selected on a validation slice of 339 examples: 323 with a trained fingerprint and 16 with a fingerprint that occurred nowhere else. Sixteen is small, and we read that column only as a range.

The held-out set began as 225 problems from book types absent from training. These carry 16 fingerprints, and one scheduling type contributes 100 of the 225, so a count out of 225 is closer to a count over 16 programs than over 225 independent problems. The set later gained a withheld procedural type of 40 problems and held-out operation combinations. For each problem the selected

checkpoint generated one program greedily; the runtime executed it; and the outcome was classified as a parse failure, an invalid graph, an execution error, an executed answer that did not match the reference string, or a match. We re-executed every saved program of the held-out evaluations of trained checkpoints (13,330 records in 26 runs) through the evaluation code, and every recorded outcome and answer was reproduced.

Sections 5 to 8 add evaluation-only comparisons that required no further training. Every model and condition answered the same items:

- 115 of the held-out problems, at most five per problem family, chosen by a seeded hash; on the book side this gives 20 coalition problems, 5 each from six other families, and 10 single problems from ten small families;
- 45 items per wording condition for the rewording tests, three per held-out or trained composition, the same instances at every wording;
- 30 book problems per wording for the shape tests;
- all 40 problems of the withheld procedural type.

Untuned base models were evaluated in three ways. They answered directly, wrote the body of one plain JavaScript function with one solved example, or wrote SOP Lang taught in the prompt by a syntax guide and three solved examples of trained types. Their JavaScript ran in the same sandbox as the fine-tuned models' programs.

These comparisons are scored at three levels: exact matching; a strict value check on the numbers and any verdict; and a semantic level at which an answer counts only if two blind judges, separate agent sessions that saw only the statement, the reference, and the candidate, both accept it. The judges agreed on 98.2% of 2,019 distinct answers (Cohen's $\kappa = 0.96$). They accepted none of 20 injected wrong answers and all 50 answers that a deterministic parser proves equivalent to the reference. We report the semantic level unless stated otherwise.

Matching compared normalized answer strings, which can reject a correct answer phrased differently. For the early runs this explains almost none of the failures. Among the book-type answers that executed but did not match, no more than 6 in any early run even contained every number of the reference; the rest either failed to execute or returned different values. Where later runs are affected, we say so and report a retrospective parser alongside the exact count.

3. A complete language and no procedure

Table 1 shows the first result, which set the question for everything that followed.

Table 1. The coverage ladder in the first series (0.5B model). Validation: 323 examples with trained programs and 16 with untrained programs (selected checkpoint). Held-out: 225 problems from book types absent from training (265 for the low-rank run).

Run	Trained programs correct	Untrained programs correct	Held-out types correct	Held-out programs that executed
Full fine-tuning, learning rate 2×10^{-5}	56.0%	1 of 16	0.0%	12.4%
Full fine-tuning, learning rate 10^{-4}	98.8%	2 of 16	0.0%	27.6%
Low-rank adaptation (rank 16)	54.2%	3 of 16	0.0%	12.5%

The untuned base model could not produce the language at all: none of 50 sampled problems yielded a parseable program. After fine-tuning at the higher learning rate, every held-out program parsed and formed a valid graph. Problems with trained programs were solved almost always, new programs of trained types rarely, and untrained types never. The lower learning rate left the model far from convergence on its own training data and produced the same zero. The failure was therefore not an artefact of a recipe that tuning could fix.

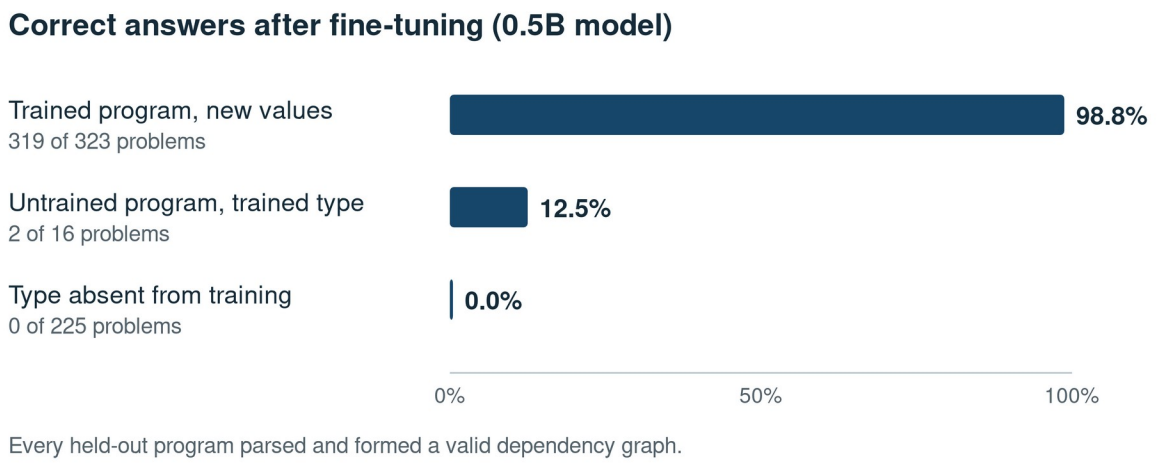


Fig. 2 The coverage ladder of the best first-series run. Validation problems with trained programs (323) and untrained programs (16), and held-out problems from untrained types (225, from 16 programs)

The failing programs show what the model did instead. Consider the held-out scheduling problem as the model saw it:

> Scenario. In household faults and maintenance observations, five work packages A–E must be completed. A takes 7 minutes. B (11 min) and C (17 min) can start only after A but may then run in parallel. D (6 min) needs both B and C finished. E (12 min) follows D. A final safety or review buffer of 4 minutes is mandatory, and the completion limit is 43 minutes. The description mentions the total number of observations, but the task durations already incorporate that workload, so recounting them would double-count input. What is the earliest safe completion time, and does the plan meet the limit?

The correct program is a chain with one join: 7 + max(11, 17) + 6 + 12 + 4 = 46 minutes, which exceeds the limit. The model's program extracted the durations correctly but declared A and C, rather than B and C, as the parallel pair. It opened with a validation line assembled from two different training programs: one checked a map of durations with `every`, the other checked a single number with `Number.isInteger`. The combination left a parenthesis unclosed, so the program did not compile. Its return sentence ended "because the total number of observations is 5". The model had reacted to the distractor sentence the statement told it to ignore.

Across the 225 held-out problems, the failures fall into three groups:

- 80 programs did not compile. In 58 of them, the spliced validation line was the cause.
- 77 compiled but were stopped by an assertion the model had written itself. Examples are "at least one group must reach the winning threshold" and "every work package must have a single duration". These are constraints the problem did not state or that the extracted values violated.
- 62 ran to completion and returned wrong answers. Of these, 50 came from one rate type. Those programs computed the right kind of quantity, useful output over a period, but got the units wrong: 48 of the 50 multiplied the throughput by the percentage and by the running time in

minutes, without dividing the percentage by 100 or converting minutes to hours. Some printed a template with unfilled fields, such as "The useful output is 125400 undefined in undefined minutes."

The remaining six hit other runtime errors. The pattern resembles what Dziri and colleagues describe as matching stored computation subgraphs (Dziri et al., 2023). The model assembled programs from remembered fragments, including their assertions and their sentences, rather than deriving the computation the statement required.

4. Interventions that did not move transfer to untrained types

The natural response to Table 1 was to change the training. The programme tried the interventions in Table 2 in sequence, each suggested by the previous result. The data-side changes were cumulative, each building on the previous dataset, so the rows are a sequence rather than independent ablations.

Table 2. Interventions against the held-out result (0.5B model). Untrained programs: best over all scored checkpoints of the run, out of 16. Held-out: 265 problems, the 225 book-type problems plus 40 from a withheld procedural type; the first row used the 225 only.

Intervention	Hypothesis	Untrained programs	Held-out correct	Held-out executed
Lower learning rate	Less memorization, more generalization	up to 2 of 16	0 of 225	12.4%
Low-rank adaptation	Preserving the base model preserves general skill	up to 3 of 16	0 of 265	12.5%
Wider curriculum (ten procedural types added)	More kinds of program	up to 3 of 16	1 of 265	18.1%
Multi-stage targets (intermediate wires)	Showing intermediate structure teaches decomposition	up to 3 of 16	1 of 265	12.8%
Twenty program shapes with two published stages	Deeper taught structure	up to 4 of 16	1 of 265	30.9%
Capability-preservation mixture (10%)	Restoring general code skill helps new programs	up to 4 of 16	0 of 265	19.2%
Contrastive minimal pairs	Forcing attention to the decisive word	up to 3 of 16	1 of 265	20.8%
One or three solved examples in the prompt	Demonstrations enable adaptation	—	0 of 265	9.8% (three examples)

For comparison, the full-fine-tuning run of Table 1 reached up to 3 of 16 untrained programs across its checkpoints. None of these eight interventions moved held-out correctness off the floor. The single correct answer in three of these runs was the same problem, a comparison of two groups by percentage that resembles a trained type; one other run solved a different single problem. Several interventions changed how often programs executed without changing whether they were correct. The twenty-shape curriculum raised the executing fraction from about one in eight to almost one in three, but the additional executing programs returned wrong answers. The untrained-program column stayed between 1 and 4 of 16 across 72 scored checkpoints.

Low-rank adaptation (Hu et al., 2022) merits a note because it bears on a published comparison. Biderman and colleagues found that it learns less of a target domain than full fine-tuning and forgets less outside it (Biderman et al., 2024). Here it did learn less: 54% on trained programs, against 99% for full fine-tuning at the same budget. Its score on ten short instruction and JavaScript probes, 0 of 10 against 1 of 10 for full fine-tuning and 4 of 10 untuned, is too small a sample to test the forgetting claim.

Two interventions are worth describing because they show how the failure behaves.

Contrastive minimal pairs. Six procedural types rendered the same statement twice, differing only in a decisive phrase that changed the required operation. We scored the resulting model on 24 new pairs, 8 of each kind:

- **Boundary word** ("above" against "at least"): both members answered correctly in 7 of 8 pairs.
- **Largest against smallest record:** both answered correctly in none of 8. In four of these pairs one member's program crashed, and in the rest the two members returned different wrong values.
- **Fixed amount against percentage:** both answered correctly in 1 of 8.

The distinction the model learned was local and lexical: a word that flips a comparison inside an otherwise identical program. It did not learn the distinctions that require a different operand or operation. A raw count of operator occurrences in the training programs does not favour threshold comparisons, so frequency is an unlikely explanation.

Demonstrations. Placing one or three solved problems from other types in the prompt did not produce a single correct held-out answer. It moved failures from wrong answers to crashes: execution errors rose from 168 to 239 of 265.

We also tried retries separately, on a later model (the extended-inventory 0.5B model of Section 5) and on 30 in-inventory development problems. Retrying a failed program with the full error history, up to twice, made the two crashing programs run. Both then returned wrong answers, and the same 17 of 30 problems were solved with and without retries. Olausson and colleagues found self-repair in code generation limited by the model's ability to diagnose its error (Olausson et al., 2024). Our small model could repair a crash into a running program, but not into one that answered the question.

5. A curriculum of declared compositions

5.1 Design

The procedural types until then had been designed one at a time, each a small scenario with its own program. The next curriculum was designed differently. It declared ten elementary operations and defined types as compositions of them, for example "keep the values below a threshold, take the largest, add a fixed amount". Twenty-two compositions were declared. Eighteen were rendered into training examples; four were held out whole before any example was generated, so that no training example contained them in any form. Each trained composition received enough instances to be learnable. In the same data version, the validation scaffolding that every target program had carried moved into the runtime. From then on, target programs contained only the computation and its answer assertions.

A second tranche added operations identified by counting what the book types' reference programs actually do: remainders, percentages and discounts, ranked positions such as the third-largest value, and ratios. It declared 37 compositions and held out eight: the four original ones and four that each used at least one new operation. The same count also listed operations the books needed that the

inventory still lacked, including time arithmetic, graph traversal, set operations, geometry, and probability.

5.2 Results

Table 3 shows the held-out results, and Fig. 3 plots them.

Table 3. Held-out results under the composition curriculum. Each held-out composition has 40 problems; no training example contains it. Held-out compositions have depths of three or four operations.

Held-out slice	0.5B, first inventory	0.5B, extended inventory	1.5B, extended inventory
Unseen compositions of the original operations (4 compositions)	160 of 160	160 of 160	160 of 160
Unseen compositions including a newly added operation (4)	—	96 of 160	160 of 160
Withheld procedural type outside the inventory	0 of 40	0 of 40	0 of 40
Book types outside the inventory (16 programs)	1 of 225	2 of 225	2 of 225

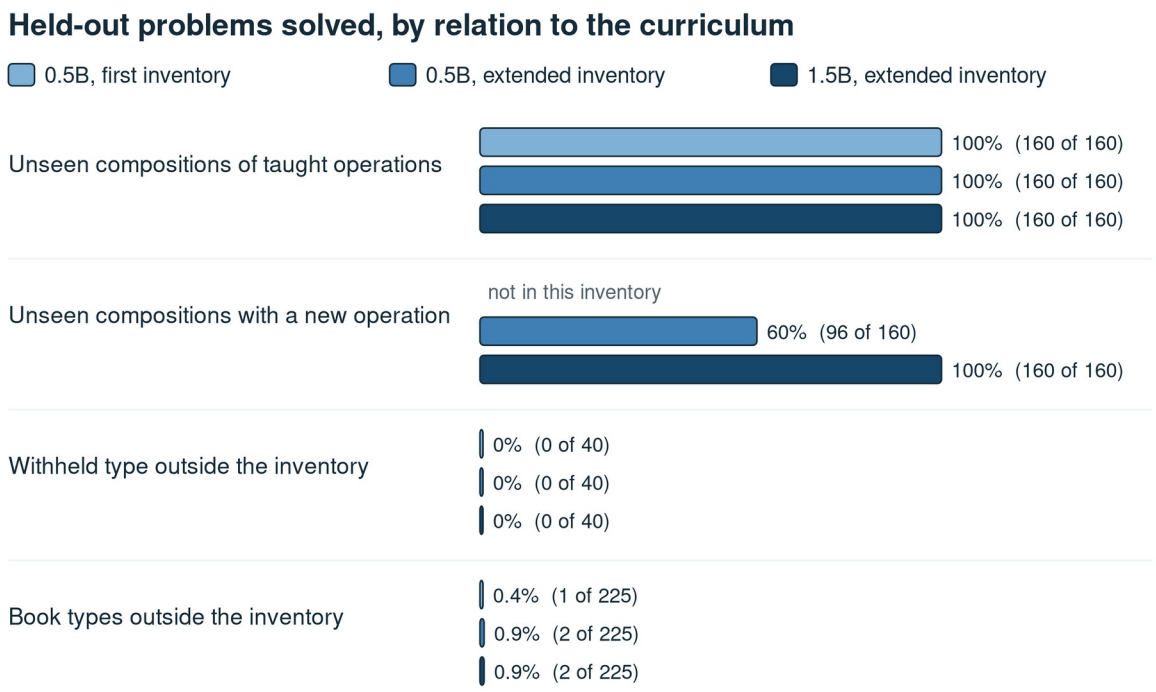


Fig. 3 Held-out results by relation to the curriculum. Each held-out composition contributes 40 problems, none of which appeared in training in any rendering. The first inventory did not contain the newly added operations

The first row is the programme's first transfer result above floor level. The model solved every instance of four compositions it had never seen in any form. There is a partial "before" for comparison. A diagnostic suite of similar filter-and-aggregate chains, scored on the preservation-mixture model before the inventory existed, gave 8 of 60 correct. That model scored 0 of 10 on each of four chain structures, including taking the largest kept value and adding a fixed amount, a chain

that the inventory later held out. The diagnostic suite used a different generator, so the comparison is suggestive rather than paired.

Two readings of the first row must be distinguished. On the vocabulary reading, the model transferred because the held-out compositions used only operations it had been taught as composable units. On the template reading, it transferred because the held-out compositions were rendered by the same generator as the trained ones, with the same statement template, input names, stage structure, and answer sentence. The withheld procedural type and the book types differ from the trained compositions both in their operations and in their surface, so they cannot separate the two readings.

We separated them by rewording. The same held-out compositions, and four trained compositions as controls, were rendered at three distances from the generator, with the numbers, operations, and reference answers unchanged:

- **Original:** the generator's own statement.
- **Paraphrase:** the same content and order in different words, for example "Discard every entry of 25 or less" for "keep only the records above 25".
- **Narrative:** the style of the book problems, with a scenario, the operations described in prose, one irrelevant number, and the question last.

Table 4 shows the result for three models, each scored on the same problems at every wording: three instances per composition. The two inventory-phase runs were trained before the time, path, and area operations existed, so their rows use only the eight held-out compositions and three controls within their own inventory.

Table 4. Rewording test (semantic level; exact matches in parentheses). The same instances at every wording; numbers and references unchanged.

Model	Wording	Held-out compositions	Trained controls
0.5B, extended inventory	Original	18 of 24 (18)	9 of 9 (9)
0.5B, extended inventory	Paraphrase	4 of 24 (4)	5 of 9 (5)
0.5B, extended inventory	Narrative	5 of 24 (3)	6 of 9 (5)
1.5B, extended inventory	Original	24 of 24 (24)	9 of 9 (9)
1.5B, extended inventory	Paraphrase	16 of 24 (4)	7 of 9 (2)
1.5B, extended inventory	Narrative	8 of 24 (4)	2 of 9 (0)
1.7B, compact targets	Original	33 of 33 (33)	12 of 12 (12)
1.7B, compact targets	Paraphrase	18 of 33 (13)	9 of 12 (8)
1.7B, compact targets	Narrative	24 of 33 (20)	11 of 12 (9)

Rewording hurt compositions the models had been trained on, as well as held-out ones, so part of the transfer in Table 3 is bound to the generator's wording rather than to the operations. It hurt the held-out compositions more. At 0.5B a paraphrase left 4 of 24 held-out problems solved against 5 of 9 trained ones. The 1.7B run kept 11 of 12 trained compositions in narrative wording, but only 24 of 33 held-out ones. Both readings therefore hold in part. New compositions of taught operations transfer only in familiar wording, and even familiar compositions degrade when the wording changes, at 0.5B and 1.5B more than in the 1.7B run. That run differs from the other two in model family and training

data as well as size, so we do not attribute its robustness to size alone. The counts are small, three instances per composition, and the direction is consistent across the three runs. The book types are worded like the narrative condition, which even trained compositions did not reliably survive at the two smaller sizes.

The second row shows first exposure to a new operation. Of the four compositions that included one, the 0.5B model solved two completely (remainders and ratios) and two poorly: 7 and 9 of 40, for a percentage discount and for a percentage of the third-largest value. The failures were uniform: the model used the new operation as a variant of its neighbour. In the third-largest composition, the programs indexed the sorted list with the percentage instead of the rank, which they never extracted. In the discount composition, they subtracted the discount multiplied by 100, a fragment of the percentage operation applied in the wrong place. In all 64 failures the program's final assertion, that the answer must be a non-negative whole number, caught the wrong value.

The 1.5B model trained on the same data solved all eight held-out compositions. This first-exposure success did not replicate across data versions. A 1.5B model trained on the next data version, with the same base, solved 17 and 0 of 40 on the same two compositions and failed the same way; a 0.5B model on that version solved neither. A later 1.5B run solved both. First-exposure transfer to a new operation was therefore partial and unstable, and it varied with the data version as much as with model size.

The third and fourth rows show that transfer outside the inventory stayed near zero. The withheld procedural type asked whether stock would last until a delivery. It required rounding days down before comparing them with a lead time. The models computed a fractional number of days and then reversed the verdict, answering "The stock covers 18.642857142857142 whole days and must be placed today" where the reference reads "The stock covers 18 whole days, so the order can wait." The validation column of untrained programs did not move either: it stayed at 3 or 4 of 16 in the three inventory runs.

5.3 Teaching the missing shapes

Later runs on Qwen3-1.7B extended the inventory with procedural types modelled on specific book shapes, rendered with different wording and numbers. Two of these shapes correspond to held-out book types.

Minimal winning coalitions. Given parties and seat counts, list the smallest groups that reach a majority. The book held-out set contained 20 problems of this shape. Every run before the procedural type entered training solved 0 of 20, mostly with programs that failed to execute. In the first 1.7B run that included it, all 20 were solved. Every later 1.7B run solved at least 16 of 20. In one of those runs the model separated coalitions with commas where the reference used semicolons, and a comparison of coalition sets rather than strings was needed to see this. The 0.5B model trained on the same final data solved 0 of 20; all of its programs failed to compile.

Scheduling with a join. For the 100 held-out problems of the type in Section 3, exact matching accepted none in any run. A retrospective parser checked the stated duration and verdict. We wrote it after seeing the outputs, and it accepts the reference sentence template and the later compressed reference form. It found 64 of 100 correct in the first 1.7B run that included the shape. In the six 1.7B runs trained with the shape, the correct counts were 64, 0, 0, 2, 54 and 0 of 100, and the 0.5B run on the final data found 3. Many of the remaining outputs failed to execute, though not in every run. The parser is strict: in the second of these runs, 18 outputs state the correct duration and verdict in

wordings it does not accept ("does not meet the completion limit"), so with a looser grammar three of the six 1.7B runs show transfer.

Whether these failures reflect the plan or the wording can be tested directly. We rewrote 15 held-out scheduling problems and 15 coalition problems into the sentence template of the procedural types that teach those shapes. The numbers and reference answers were kept, so the rewritten scheduling problems still carried durations outside the range the procedural generator had produced. Table 5 gives the result.

Table 5. Book problems in their own wording and in the wording of the procedural type that teaches their shape (semantic level, 15 problems per cell). The first run was never taught either shape.

1.7B run	Scheduling, book wording	Scheduling, procedural wording	Coalitions, book wording	Coalitions, procedural wording
Before the shapes were taught	0	0	0	0
Compact targets	15	15	15	15
Simplified statements	8	15	7	15
Final data	0	15	15	15

Every run taught the shapes solved every rewritten problem, including the final-data run, which solved none of the same scheduling problems in their book wording. The run never taught the shapes solved none in either wording. The scheduling plan had been learned. What failed in most later runs was recognizing it under the book's wording.

Teaching the solution shape through independently generated problems was therefore necessary in these data: no run solved either type before its shape was taught. Whether a run then solved the book problems depended on how closely the book's wording resembled what the run had been trained on. The coalition shape transferred to book wording in almost every later run of the 1.7B model. The scheduling shape transferred in two or three of six, although every one of them solved it in procedural wording. Neither transferred at 0.5B.

6. What fine-tuning adds over writing plain code

A reviewer will ask whether any of this requires fine-tuning. A code model can already write a program that computes an answer and let an interpreter run it (Gao et al., 2023; Chen et al., 2023). Table 6 compares the untuned bases in three prompted conditions with the fine-tuned students on the same 115 held-out problems.

Table 6. Correct answers on the same held-out problems (semantic level): 55 problems from the 11 held-out compositions, 55 from the book types. JavaScript: one solved example of a trained type. SOP Lang in the prompt: a syntax guide and three solved examples of trained types. Fine-tuned: the final-data 0.5B run, the 1.5B run on the declarative-command data, and the 1.7B compact-target run.

Model	Condition	Held-out compositions (of 55)	Book types (of 55)
0.5B	Untuned, direct answer	9	5
0.5B	Untuned, plain JavaScript	33	1
0.5B	Untuned, SOP Lang in prompt	4	2
0.5B	Fine-tuned SOP Lang	45	4
1.5B	Untuned, direct answer	2	13
1.5B	Untuned, plain JavaScript	49	10
1.5B	Untuned, SOP Lang in prompt	19	9
1.5B	Fine-tuned SOP Lang	55	6
1.7B	Untuned, direct answer	17	11
1.7B	Untuned, plain JavaScript	45	10
1.7B	Untuned, SOP Lang in prompt	34	1
1.7B	Fine-tuned SOP Lang	55	32

Three observations follow.

- **Most of the gain on the held-out compositions comes from executing code, not from fine-tuning.** Answering directly, the bases solved between 2 and 17 of the 55. Writing plain JavaScript with a single example, the 1.5B and 1.7B bases solved 49 and 45. Fine-tuning raised this to 55 at both sizes, and from 33 to 45 at 0.5B.
- **SOP Lang taught only in the prompt was worse than plain JavaScript at every size.** An unfamiliar language costs a small model more than the structure it offers, until the model is trained on it.
- **The book types separate the conditions.** No untuned base solved more than 13 of the 55 in any condition. The fine-tuned 1.7B compact-target model solved 32 (the final-data run 27), 20 of them the coalition problems whose shape its curriculum taught; coalitions are over-represented in the sample relative to the full book set. At the two smaller sizes, fine-tuning did not help on book types.

What fine-tuning adds, then, is not the ability to compose operations, which a code model with one example largely has. It adds reliable execution of the taught shapes, including shapes that plain code-writing does not reach, and, as Section 8 shows, it takes something away.

7. What scale changed

The same extended curriculum was used to train the 0.5B and the 1.5B model of the same family with the same recipe. Within the inventory the larger model was better: its procedural accuracy rose from 71% to 89%, and it solved the first-exposure compositions the smaller model had failed, though

Section 5.2 shows this did not hold in another data version. Outside the inventory it was no better. It solved 2 of 225 book problems, the same number as the smaller model, one of them the same problem, and none of the withheld procedural type. More of its book-type programs ran: execution errors fell from 195 to 154 of 225. The additional programs that ran returned wrong answers.

On the final data version, the 1.7B Qwen3 model solved the taught coalition shape and the 0.5B Qwen2.5-Coder model did not. Model family and pretraining differ in that pair, so it shows a system difference rather than a size effect. For the sizes tested, the larger models did better on what the curriculum covered and, in one comparison, on a taught book shape. None of them transferred to what it did not cover.

8. What the narrow curriculum cost and bought

Specialization had a cost that matters for deployment. On ten short capability probes, half instruction-following and half JavaScript, the untuned 0.5B model passed 4. Fine-tuned 0.5B versions passed between 0 and 2, and a 10% mixture of general code did not restore them. This fits the trade-off Fu and colleagues describe for specialized small models (Fu et al., 2023) and the broader pattern of forgetting under sequential training (Kirkpatrick et al., 2017). The three 1.5B runs passed 5, 1, and 3 of 10. The untuned 1.5B model was not probed, and with ten probes these differences are within noise.

The evaluation-only comparisons show a larger cost. The withheld procedural type asks whether stock covers the days until a delivery. In their original evaluations the fine-tuned models solved almost none of its problems (1 of 40, in one 0.5B run); re-evaluated on all 40 and scored semantically, the two fine-tuned 1.7B runs solved 2 and 0. The untuned Qwen3-1.7B base, writing plain JavaScript with one example of an unrelated trained type, solved 24 of the 40. For example, it returned "The stock covers 13 whole days. The order must be placed today: No." where the reference reads "The stock covers 13 whole days, so the order can wait." Under narrative wording, the same untuned base writing JavaScript also matched the fine-tuned 1.7B model on held-out compositions (26 against 24 of the same 33 problems; the fine-tuned figure is from Table 4). Fine-tuning on templated synthetic programs made the model more accurate within its templates and less able to handle problems outside them, including problems the base model could solve by writing ordinary code. The untuned model wrote JavaScript while the fine-tuned models wrote SOP Lang from a different prompt, so this comparison does not isolate fine-tuning from the change of target language; a matched fine-tuning comparison was not run. What it shows is that the fine-tuned system, as built, could not do what its starting model could do by writing ordinary code. The untuned 1.5B code model, by contrast, solved only 2 of the 40.

The compiled path gained capabilities the direct path lacked. On a suite of 49 text-reasoning probes (counting letters, counting words, reversing words, ordering), the fine-tuned model answered none correctly when asked to answer directly, and the untuned base answered 5. The same fine-tuned model answered 30 when it compiled each probe into a program and the runtime executed it. These included all 16 letter-counting and all 9 word-counting probes, whose programs the curriculum had taught.

9. Discussion

9.1 The answer to the question

A small model fine-tuned to write programs learned three things with very different reach:

- It learned the language completely and at once.
- It learned the programs it was shown, recalling them for new values, and it recombined taught operations into unseen compositions when they were worded like its training data. Rewording broke new compositions more than trained ones: at 0.5B a paraphrase was enough, while the 1.7B run kept more than half.
- It did not produce working programs for problem types whose solution shape the curriculum lacked. No change to optimization, adapter, data mixture, target structure, prompt, retries, or model size within the tested range produced such programs.

When a missing shape was taught through independently generated problems, the model learned the plan, and it applied the plan reliably when the problem was worded like the teaching data. Under the book's own wording it often did not. Untuned models writing plain JavaScript already composed the elementary operations almost as well, and the untuned 1.7B model solved most problems of a type the fine-tuned models rarely solved.

This matches the account of compositional generalization in which unseen compounds of familiar atoms are handled and unfamiliar atoms are not (Keysers et al., 2020; Bogin et al., 2022). Our additions are specific to the practical setting. A half-billion-parameter model handled unseen compounds of familiar operations essentially perfectly, but only in its training generator's wording. For the two book shapes we reworded, it was this binding, more than the missing operations, that separated success from failure. A newly introduced operation was first used as a variant of its nearest neighbour. And recipe, presentation, and inference-time levers, each well motivated, did not substitute for coverage. The results concern systematic recombination within a closed set of operations. They do not test productivity in Hupkes and colleagues' sense of generalizing to longer inputs (Hupkes et al., 2020), and failure on unseen operations lies outside their taxonomy.

9.2 What to generate

The practical rule is to treat the curriculum as a statement of coverage:

- Measure which operations and solution shapes the target problems require and which the curriculum teaches.
- Teach each operation in enough compositions to be composable, placed beside the operations it could be confused with.
- Measure transfer by holding out whole compositions and whole problem types.
- Vary the wording of every taught shape as deliberately as its numbers, because the wording is what a small model binds to.
- Compare against the untuned base writing plain code, which is a strong baseline for elementary operations and, outside the curriculum, may be stronger than the fine-tuned model.

More instances of covered programs buy recall of those programs. The contrastive and demonstration results suggest that clever presentation of covered programs does not buy reach beyond them. When a problem type is missing, teaching its shape is the lever that moved results in our data, and whether that transfers to differently worded problems must still be measured per shape and per model.

9.3 More data, or different data?

It is natural to ask whether the models simply had too few problems. Our records answer part of the question and the literature suggests the rest.

What our data show.

- **Volume within covered types did not help.** Widening the plan set, adding a preservation mixture, and adding contrastive rows grew the training set from about 6,800 to more than 8,000 rows. The untrained types stayed at the floor, and the untrained-program column did not move.
- **Diversity of operations helped, and at modest volume.** Forty instances per composition were enough for complete transfer to unseen compositions in the generator's wording.
- **Diversity of wording appears to be the missing dimension.** The rewording and teaching-template tests show the same plans succeeding in one wording and failing in another. We did not train with varied wordings, so the dose of wording diversity that would be needed is not something our data can estimate.

What the literature adds. Studies of compositional generalization point the same way:

- Selecting structurally diverse synthetic examples matched the effect of a training set two hundred times larger on a semantic-parsing benchmark (Oren et al., 2021).
- At fixed training-set size, more distinct primitives turned a failing sequence model into a near-perfect generalizer on a controlled task (Patel et al., 2022).
- Demonstrations chosen to cover the needed output structures help in-context compositional generalization (Levy et al., 2023). Our own demonstrations, chosen from other types, did not.
- For alignment rather than new capabilities, a small, diverse, curated set was shown to rival much larger ones (Zhou et al., 2023).
- Distilling many sampled reasoning chains into small students improves their reasoning (Li et al., 2023).

Together these suggest that the productive next step is a structural inventory of operations and wordings, not more rows of the same kind.

Would more data degrade the model? Our fine-tuned models lost capabilities: short instruction and code probes fell, and a problem type that the untuned model solved by writing code became unsolvable. The literature documents the same trade-off:

- concentrating a small model's capacity on a target task costs generic abilities (Fu et al., 2023);
- continual instruction tuning of models from 1B to 7B parameters makes them forget general knowledge and reasoning, and within that range the forgetting grew with size (Luo et al., 2025);
- Kotha and colleagues argue that capabilities lost to narrow fine-tuning are often suppressed rather than erased, and partly recover them through changes to the prompt (Kotha et al., 2024).

The rows that caused our losses were narrow, not numerous. Broader and more varied data may reduce the loss, but we did not measure that. The claim that some abilities appear only above a certain scale (Wei et al., 2022) is a reminder that no amount of curriculum design is guaranteed to substitute for capacity. The comparison with larger models in Section 9.4 shows that most of the wording dependence is a matter of capacity.

What follows for the approach. In our data, small fine-tuned models were reliable within a bounded class of problems whose operations and wordings the curriculum covers. Pushing them towards broad coverage by fine-tuning alone ran into two limits: an inventory of operations and wordings that grows with the domain, and a loss of the general abilities that let the untuned model cope outside it. We read this as a negative result for broad coverage by small models alone. The larger models in Section 9.4, prompted with the same language, had neither limit. It is not a negative result for small models in a system that routes problems outside their taught classes to a more capable model.

9.4 What larger models do on the same problems

To locate the limits we found, we gave two larger models the same problems, instructions, and examples as the untuned small models, delivered in batches: Claude Haiku 4.5 and Claude Sonnet 5, both without fine-tuning. Their programs ran in the same sandbox and were scored with the same comparators and judges.

These models were evaluated through agent sessions rather than independent API calls. Each session answered a batch of items and was instructed not to execute code or read other files. We audited every session's tool use. Batches that mixed two wordings of the same problem, and one batch whose replies were assembled by a script, were discarded and re-run so that no session saw two wordings of a problem. Four of the accepted batches still paired two answer forms of the same problems (operator plan with JavaScript, or JavaScript with SOP Lang), so those forms are not independent answers: on the book problems of Section 5.3, the JavaScript and SOP Lang answers of each model were identical in all 30 cases. The larger models' programs rarely matched the reference wording exactly, so their scores rest mostly on the judges; their answers were judged in a third pass that contained no calibration items, where the two judges agreed on 616 of 619 answers.

Table 7. Larger models and small models on the same problems (semantic level). Main comparison: 55 held-out compositions and 55 book problems. Withheld type: all 40 problems. Rewording: 33 held-out compositions in generator and narrative wording.

Model	Condition	Compositions	Book types	Withheld type	Rewording, generator / narrative
1.7B untuned	Plain JavaScript	45	10	24	26 / 26
1.7B fine-tuned (compact targets)	SOP Lang	55	32	2	33 / 24
Haiku 4.5, untuned	Direct answer	48	41	—	—
Haiku 4.5, untuned	Plain JavaScript	55	52	40	33 / 33
Haiku 4.5, untuned	SOP Lang in prompt	55	47	40	31 / 33
Sonnet 5, untuned	Direct answer	55	53	—	—
Sonnet 5, untuned	Plain JavaScript	55	54	40	33 / 33
Sonnet 5, untuned	SOP Lang in prompt	55	53	40	33 / 33

The larger models solved nearly every problem in every condition:

- They were unaffected by rewording.
- They solved all 15 book scheduling problems in the book's own wording, which the final-data fine-tuned model never solved.
- They solved all 40 problems of the withheld type.

The wording binding and the coverage boundary we measured are therefore limits of the small models' capacity, not of the task or of the program-writing formulation. Code helped Haiku, the lower-cost of the two larger models: it rose from 48 and 41 answering directly to 55 and 52 writing JavaScript. Sonnet did not need it. Neither larger model gained accuracy from SOP Lang over plain JavaScript; Haiku lost some (47 against 52 book problems).

The comparison also bounds what fine-tuning bought. The best fine-tuned 1.7B model matched the larger models on the compositions its curriculum covered and fell far short everywhere else. Two caveats apply. We cannot exclude that the larger models saw material similar to these problems during pretraining. And a batched agent session is not identical to an independent API call.

9.5 Threats to validity

- **Single runs.** Each condition was trained once with one seed. We rely on contrasts large enough to be read without inferential statistics, such as 0 against 160 of 160. Where replication across data versions exists, we report it, and it showed first-exposure transfer to be unstable.
- **Template dependence.** The rewording test (Table 4) shows that the central positive result depends on the training generator's wording; it uses three instances per composition, the same at every wording.
- **Changes coinciding with the inventory.** The inventory data version also removed repeated validation scaffolding from the targets.
- **Small effective sample.** The held-out book set covers 16 programs, one contributing 100 problems. The untrained-program validation column has 16 items.
- **Adaptive reuse.** The held-out book set was reused to guide decisions throughout, so it is a development benchmark rather than a clean test (Dwork et al., 2015). This matters more for later runs than for the first use of each held-out composition.
- **Answer matching.** Exact matching under-counts correct answers for some later book-type outputs. We report a retrospective parser where it matters, and the parser was written after seeing the outputs.
- **Evaluation-only samples.** The comparisons of Sections 5 to 8 use five problems per family or composition (all 40 for the withheld type) and one greedy decode. Their semantic level rests on two agent judges, whose agreement and calibration are reported in Section 2.3.
- **Prompted baselines.** The untuned bases were prompted with one or three examples of trained types, not tuned. A matched fine-tuning comparison of plain JavaScript and SOP Lang targets was not run.
- **Scope.** The operations are elementary and synthetic.

10. Conclusion

We set out to teach a small model to write programs for problems it had not seen. It learned the language at once, learned the programs it was shown, and recombined their operations, but only in the wording it was trained on. It could not write programs for problem types whose solution shape it had not been taught, and recipe, data presentation, prompting, retries, and model size in the tested range did not change that. Teaching the missing shape made the plan available, but the model used it reliably only when the problem was worded like the teaching data. Meanwhile, untuned models composed elementary operations almost as well by writing plain code, and the untuned 1.7B model solved most problems of a type the fine-tuned models rarely solved. For anyone generating training data for a small program-writing model:

- Decide which operations and solution shapes the model must handle.
- Teach each one as a composable unit, next to the operations it could be confused with.
- Vary the wording as deliberately as the numbers, and measure transfer on whole compositions and problem types held out from training, rendered in more than one style.

- Report the untuned model writing plain code as the baseline to beat, and a larger model as the ceiling.

On the evidence here, small fine-tuned program writers suit bounded problem classes inside a system that sends everything else to a larger model.

Data and code availability

The runtime, data generators, training and evaluation code, per-item evaluation records, run manifests, and the scripts that recompute every number in this paper from those records are provided as supplementary material, including a re-execution of every saved held-out program. A public archive will be deposited on acceptance. A crosswalk from the conditions named here to the archived run identifiers is included. Coding agents assisted in building the software and in drafting this manuscript; the authors verified every reported result against the item records.

References

Biderman, D., Portes, J., Gonzalez Ortiz, J. J., Paul, M., Greengard, P., Jennings, C., King, D., Havens, S., Chiley, V., Frankle, J., Blakeney, C., & Cunningham, J. P. (2024). LoRA Learns Less and Forgets Less. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=aloEru2qCG

Bogin, B., Gupta, S., & Berant, J. (2022). Unobserved Local Structures Make Compositional Generalization Hard. *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2731-2747. https://doi.org/10.18653/v1/2022.emnlp-main.175

Chen, W., Ma, X., Wang, X., & Cohen, W. W. (2023). Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=YfZ4ZPt8zd

Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. (2015). The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248), 636-638. https://doi.org/10.1126/science.aaa9375

Dziri, N., Lu, X., Sclar, M., Li, X. L., Jiang, L., Lin, B. Y., Welleck, S., West, P., Bhagavatula, C., Le Bras, R., Hwang, J. D., Sanyal, S., Ren, X., Ettinger, A., Harchaoui, Z., & Choi, Y. (2023). Faith and Fate: Limits of Transformers on Compositionality. *Advances in Neural Information Processing Systems*, 36, 70293-70332. https://doi.org/10.52202/075280-3081

Fu, Y., Peng, H., Ou, L., Sabharwal, A., & Khot, T. (2023). Specializing Smaller Language Models towards Multi-Step Reasoning. *Proceedings of Machine Learning Research*, 202, 10421-10430. https://proceedings.mlr.press/v202/fu23d.html

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2023). PAL: Program-aided Language Models. *Proceedings of Machine Learning Research*, 202, 10764-10799. https://proceedings.mlr.press/v202/gao23f.html

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations*. https://openreview.net/forum?id=nZeVKeeFYf9

Hupkes, D., Dankers, V., Mul, M., & Bruni, E. (2020). Compositionality Decomposed: How do Neural Networks Generalise? *Journal of Artificial Intelligence Research*, 67, 757-795. https://doi.org/10.1613/jair.1.11674

Keysers, D., Schärli, N., Scales, N., Buisman, H., Furrer, D., Kashubin, S., Momchev, N., Sinopalnikov, D., Stafiniak, L., Tihon, T., Tsarkov, D., Wang, X., van Zee, M., & Bousquet, O. (2020). Measuring Compositional Generalization: A Comprehensive Method on Realistic Data. *International Conference on Learning Representations*. https://openreview.net/forum?id=SygcCnNKwr

Kim, N., & Linzen, T. (2020). COGS: A Compositional Generalization Challenge Based on Semantic Interpretation. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 9087-9105. https://doi.org/10.18653/v1/2020.emnlp-main.731

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521-3526. https://doi.org/10.1073/pnas.1611835114

Kotha, S., Springer, J. M., & Raghunathan, A. (2024). Understanding Catastrophic Forgetting in Language Models via Implicit Inference. *International Conference on Learning Representations*. https://openreview.net/forum?id=VrHiF2hsrm

Lake, B., & Baroni, M. (2018). Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks. *Proceedings of Machine Learning Research*, 80, 2873-2882. https://proceedings.mlr.press/v80/lake18a.html

Levy, I., Bogin, B., & Berant, J. (2023). Diverse Demonstrations Improve In-context Compositional Generalization. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1401-1422. https://doi.org/10.18653/v1/2023.acl-long.78

Li, L. H., Hessel, J., Yu, Y., Ren, X., Chang, K.-W., & Choi, Y. (2023). Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2665-2679. https://doi.org/10.18653/v1/2023.acl-long.150

Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., & Zhang, Y. (2025). An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-Tuning. *IEEE Transactions on Audio, Speech and Language Processing*, 33, 3776-3786. https://doi.org/10.1109/TASLPRO.2025.3606231

Magister, L. C., Mallinson, J., Adamek, J., Malmi, E., & Severyn, A. (2023). Teaching Small Language Models to Reason. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1773-1781. https://doi.org/10.18653/v1/2023.acl-short.151

Olausson, T. X., Inala, J. P., Wang, C., Gao, J., & Solar-Lezama, A. (2024). Is Self-Repair a Silver Bullet for Code Generation? *International Conference on Learning Representations*. https://openreview.net/forum?id=y0GJXRungR

Oren, I., Herzig, J., & Berant, J. (2021). Finding needles in a haystack: Sampling Structurally-diverse Training Sets from Synthetic Data for Compositional Generalization. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 10793-10809. https://doi.org/10.18653/v1/2021.emnlp-main.843

Patel, A., Bhattamishra, S., Blunsom, P., & Goyal, N. (2022). Revisiting the Compositional Generalization Abilities of Neural Sequence Models. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 424-434. https://doi.org/10.18653/v1/2022.acl-short.46

Qwen. (n.d.-a). *Qwen2.5-Coder-0.5B-Instruct* [Model card]. Hugging Face. Retrieved September 28, 2026, from https://huggingface.co/Qwen/Qwen2.5-Coder-0.5B-Instruct

Qwen. (n.d.-b). *Qwen2.5-Coder-1.5B-Instruct* [Model card]. Hugging Face. Retrieved September 28, 2026, from https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B-Instruct

Qwen. (n.d.-c). *Qwen3-1.7B* [Model card]. Hugging Face. Retrieved September 28, 2026, from https://huggingface.co/Qwen/Qwen3-1.7B

Shi, K., Hong, J., Zaheer, M., Yin, P., & Sutton, C. (2022). Compositional Generalization and Decomposition in Neural Program Synthesis. *Deep Learning for Code (DL4C) Workshop at ICLR 2022*. arXiv:2204.03758. https://arxiv.org/abs/2204.03758

Shi, K., Hong, J., Deng, Y., Yin, P., Zaheer, M., & Sutton, C. (2024). ExeDec: Execution Decomposition for Compositional Generalization in Neural Program Synthesis. *The Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=oTRwljRgiv

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., & Fedus, W. (2022). Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=yzkSU5zdwD

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., & Levy, O. (2023). LIMA: Less Is More for Alignment. *Advances in Neural Information Processing Systems*, 36, 55006-55021. https://doi.org/10.52202/075280-2400